

13601-02, 13601-03: 알고리즘

Spring 2026, School of Computer Science and Engineering

Practice 4 – 백트래킹

20235685 최세현

Description: 주어진 무방향 그래프에 대해 인접한 두 정점이 같은 색을 갖지 않도록 m 개의 색으로 모든 정점을 색칠할 수 있는지 확인하는 그래프 색칠 문제를 풀이합니다. 문제를 풀이할 때는 다음의 세 가지 알고리즘을 사용합니다.

1. **완전 탐색:** 모든 색 배정을 brute-force 방식으로 검사합니다.

2. **기본 백트래킹:** 그래프에서 정점 번호 순서대로 색을 배정하며, 이미 색칠된 인접 정점과 충돌하면 가지치기합니다. 즉, 상태 공간 트리의 k 번째 레벨에서는 k 번 정점의 색을 결정합니다.

3. **개선 백트래킹:** 그래프에서 차수가 높은 정점부터 색을 배정하며, 이미 색칠된 인접 정점과 충돌하면 가지치기합니다. 즉, 그래프에서 인접 정점이 많은 정점을 상태 공간 트리의 얕은 레벨에서 먼저 결정합니다.

기본 백트래킹은 주어진 정점 순서를 그대로 따라가며 탐색하고, 개선 백트래킹은 제약을 많이 만드는 정점을 먼저 선택하여 실패할 가치를 더 이른 단계에서 발견하려는 방식입니다.

총 3가지 알고리즘에 대해 아래의 파일별로 색칠 가능 여부, 실행 시간, 방문 노드 수, 가지치기 횟수를 비교 분석하여 제출합니다.

다음은 데이터 파일의 구조입니다. 데이터 파일은 sparse_coloring.txt, dense_coloring.txt, hard_coloring.txt 세 가지로 제공됩니다. 순서대로 희소 그래프, 밀집 그래프, m -coloring에 실패하는 그래프 파일입니다.

〈데이터 파일 구조〉	〈파일 예시〉
	5 5 3
〈정점수〉 〈간선수〉 〈색상수〉	0 1
〈정점번호〉 〈정점번호〉	0 2
...	1 2
〈정점번호〉 〈정점번호〉	1 3
	2 4

비교 분석 시 다음 질문에 답해 보시기 바랍니다.

1. 희소 그래프와 밀집 그래프에서 방문 노드 수와 가지치기 횟수는 어떻게 달라졌는가?
2. 정점 번호 순서와 차수 내림차순 순서는 방문 노드 수에 어떤 차이를 만들었는가?
3. 유망 함수가 왜 건전해야 하는가?

데이터는 PC 환경에서 수 초 이내로 수행할 수 있는 규모이나, 연산이 오래 걸리는 경우 임의로 정점의 개수를 줄여 사용하여도 무방합니다. 다만, 보고서에 정점 수를 제한한다는 것을 반드시 기재하시기 바랍니다.

사용 언어에는 제한이 없으나 과제 지시 편의를 위해 Python을 사용할 것을 권장합니다.

분량 제한은 없으며 자유 형식으로 제출하시기 바랍니다.

1. 사용한 알고리즘

본 과제에서는 그래프 색칠 문제를 해결하기 위해 완전 탐색, 기본 백트래킹, 개선 백트래킹의 세 가지 알고리즘을 사용하였다.

1.1 완전 탐색

완전 탐색은 모든 정점에 대해 가능한 모든 색 배정을 검사하는 방식이다.

정점의 개수가 n 개이고 사용할 수 있는 색의 개수가 m 개라면, 가능한 색 배정의 수는 최대 m^n 개이다. 이 방법에서는 먼저 각 정점에 색을 모두 배정한 뒤, 마지막에 모든 간선을 확인하여 인접한 두 정점이 같은 색을 가지는지 검사한다.

따라서 구현은 단순하지만, 정점 수가 증가할수록 검사해야 할 경우의 수가 매우 빠르게 증가한다.

1.2 기본 백트래킹

기본 백트래킹은 정점 번호 순서대로 색을 하나씩 배정하는 방식이다. 즉, 0번 정점부터 시작하여 1번, 2번 정점 순서로 색을 결정한다.

색을 배정할 때마다 이미 색칠된 인접 정점과 색이 충돌하는지 확인한다. 만약 현재 정점에 배정하려는 색이 인접 정점의 색과 같다면, 그 색 배정은 더 이상 탐색하지 않고 가지치기한다.

이 방식은 모든 색 배정을 끝까지 검사하는 완전 탐색과 달리, 중간 단계에서 불가능한 경우를 제거할 수 있기 때문에 탐색량을 줄일 수 있다.

1.3 개선 백트래킹

개선 백트래킹은 정점 번호 순서가 아니라 차수가 높은 정점부터 색을 배정하는 방식이다.

차수는 한 정점에 연결된 간선의 개수를 의미하므로, 차수가 높은 정점은 더 많은 인접 정점과 색이 달라야 한다. 따라서 차수가 높은 정점을 먼저 색칠하면 제약이 큰 정점을 상태 공간 트리의 앞부분에서 처리하게 된다. 이로 인해 색 충돌이 발생할 가능성이 높은 경우를 더 이른 단계에서 발견할 수 있고, 불필요한 하위 탐색을 줄일 수 있다.

즉, 개선 백트래킹은 기본 백트래킹과 같은 가지치기 조건을 사용하지만, 정점 선택 순서를 바꾸어 탐색 효율을 높인 방법이다.

2. 작성한 코드

```
from pathlib import Path
from time import perf_counter
import statistics

def read_graph(path):
    lines = Path(path).read_text(encoding='utf-8').strip().splitlines()
    n, e, m = map(int, lines[0].split())

    edges = [tuple(map(int, line.split())) for line in lines[1:] if line.strip()]

    adj = [set() for _ in range(n)]
    for u, v in edges:
        adj[u].add(v)
        adj[v].add(u)

    return n, e, m, edges, adj

def is_valid_full(colors, edges):
    for u, v in edges:
        if colors[u] == colors[v]:
            return False
    return True

def brute_force(path):
    """
    완전 탐색:
    모든 정점에 색을 배정한 뒤, 리프 노드에서 전체 간선을 검사한다.
    """
    n, e, m, edges, adj = read_graph(path)

    colors = [-1] * n
    visited = 0
    pruned = 0
    solution = None

    start = perf_counter()

    def dfs(k):
        nonlocal visited, solution

        visited += 1

        if k == n:
            if is_valid_full(colors, edges):
                solution = colors.copy()
                return True
            return False

        for c in range(m):
            colors[k] = c

            if dfs(k + 1):
                return True

        colors[k] = -1
        return False

    possible = dfs(0)
    elapsed_ms = (perf_counter() - start) * 1000

    return possible, elapsed_ms, visited, pruned, list(range(n)), solution

def backtracking(path, improved=False):
    """
    백트래킹:
    """
```

색을 배정하는 도중 이미 색칠된 인접 정점과 충돌하면 가지치기한다.

```
improved=False: 정점 번호 순서대로 탐색
improved=True : 차수가 높은 정점부터 탐색
"""
n, e, m, edges, adj = read_graph(path)

if improved:
    order = sorted(range(n), key=lambda v: (-len(adj[v]), v))
else:
    order = list(range(n))

colors = [-1] * n
visited = 0
pruned = 0
solution = None

def safe(v, c):
    for u in adj[v]:
        if colors[u] == c:
            return False
    return True

start = perf_counter()

def dfs(level):
    nonlocal visited, pruned, solution

    visited += 1

    if level == n:
        solution = colors.copy()
        return True

    v = order[level]

    for c in range(m):
        if safe(v, c):
            colors[v] = c

            if dfs(level + 1):
                return True

            colors[v] = -1
        else:
            pruned += 1

    return False

possible = dfs(0)
elapsed_ms = (perf_counter() - start) * 1000

return possible, elapsed_ms, visited, pruned, order, solution

def run_once(path, algorithm):
    if algorithm == 'brute':
        return brute_force(path)

    if algorithm == 'basic':
        return backtracking(path, improved=False)

    if algorithm == 'improved':
        return backtracking(path, improved=True)

    raise ValueError('unknown algorithm')

def main():
    files = [
        'sparse_coloring.txt',
```

```

        'dense_coloring.txt',
        'hard_coloring.txt'
    ]

    algorithms = [
        ('brute', '완전 탐색'),
        ('basic', '기본 백트래킹'),
        ('improved', '개선 백트래킹')
    ]

    print('파일,알고리즘,가능여부,평균시간(ms),방문노드수,가지치기횟수,정점순서,색배정')

    for file_name in files:
        path = Path(file_name)

        for algorithm, label in algorithms:
            runs = [run_once(path, algorithm) for _ in range(5)]
            times = [r[1] for r in runs]

            possible, _, visited, pruned, order, solution = runs[0]

            print(
                f'{file_name},',
                f'{label},',
                f'{"가능" if possible else "불가능"},',
                f'{statistics.mean(times):.6f},',
                f'{visited},',
                f'{pruned},',
                f'"{order}",',
                f'"{solution}"'
            )

    if __name__ == '__main__':
        main()

```

3. 실행 결과

```
sehn00@sehn-pc:~/algorithm_4_1/hw4
$ python ./backtracking_coloring.py
파일,알고리즘,가능여부,평균시간(ms),방문노드수,가지치기횟수,점점순서,색배정
sparse_coloring.txt,완전 탐색,가능,362.607812,2690434,0,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]", "[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 2]"
sparse_coloring.txt,기본 백트래킹,가능,0.007788,16,9,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]", "[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 2]"
sparse_coloring.txt,개선 백트래킹,가능,0.006351,16,9,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]", "[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 2]"
dense_coloring.txt,완전 탐색,가능,337.281515,2368555,0,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]", "[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]"
dense_coloring.txt,기본 백트래킹,가능,0.008335,13,18,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]", "[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]"
dense_coloring.txt,개선 백트래킹,가능,0.006388,13,18,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]", "[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]"
hard_coloring.txt,완전 탐색,불가능,1035.513766,7174453,0,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]", "None"
hard_coloring.txt,기본 백트래킹,불가능,1.658372,3940,7881,"[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]", "None"
hard_coloring.txt,개선 백트래킹,불가능,0.018851,28,57,"[10, 3, 6, 11, 12, 13, 0, 1, 2, 4, 5, 7, 8, 9]", "None"
```

작성한 Python 코드를 sparse_coloring.txt, dense_coloring.txt, hard_coloring.txt 세 입력 파일에 대해 실행하였다.
각 알고리즘은 동일한 입력에 대해 5회씩 실행하였고, 실행 시간은 5회 실행 결과의 평균값을 사용하였다.

입력 파일	알고리즘	색칠 가능 여부	평균 실행 시간 (ms)	방문 노드 수	가지치기 횟수
sparse_coloring.txt	완전 탐색	가능	362.607812	2,690,434	0
	기본 백트래킹	가능	0.007788	16	9
	개선 백트래킹	가능	0.006351	16	9
dense_coloring.txt	완전 탐색	가능	337.281515	2,368,555	0
	기본 백트래킹	가능	0.008335	13	18
	개선 백트래킹	가능	0.006388	13	18
hard_coloring.txt	완전 탐색	불가능	1035.513766	7,174,453	0
	기본 백트래킹	불가능	1.658372	3,940	7,881
	개선 백트래킹	불가능	0.018851	28	57

완전 탐색은 가능한 색 배정을 모두 확인하는 방식이므로 가지치기 횟수는 0으로 측정되었다.
반면 기본 백트래킹과 개선 백트래킹은 색을 배정하는 과정에서 인접 정점과 색이 충돌하면 해당 가지를 더 이상 탐색하지 않으므로, 완전 탐색에 비해 방문 노드 수와 실행 시간이 크게 감소하였다.
특히 hard_coloring.txt에서는 색칠 가능한 해가 존재하지 않기 때문에 가능한 경우를 더 넓게 확인해야 한다.
기본 백트래킹은 3,940개의 노드를 방문하고 7,881번의 가지치기를 수행한 반면, 개선 백트래킹은 28개의 노드만 방문하고 57번의 가지치기를 수행하였다.
이를 통해 차수가 높은 정점을 먼저 선택하는 방식이 불가능한 경우를 더 빠르게 발견하는 데 효과적임을 확인할 수 있다.

4. 결과 분석

4.1 희소 그래프와 밀집 그래프의 방문 노드 수 및 가지치기 횟수

희소 그래프인 sparse_coloring.txt에서는 기본 백트래킹과 개선 백트래킹 모두 방문 노드 수가 16개, 가지치기 횟수가 9회로 측정되었다.
반면 밀집 그래프인 dense_coloring.txt에서는 두 백트래킹 알고리즘 모두 방문 노드 수가 13개, 가지치기 횟수가 18회로 측정되었다.
즉, 밀집 그래프에서는 희소 그래프보다 방문 노드 수가 더 적고, 가지치기 횟수는 더 많았다. 이는 밀집 그래프가 더 많은 간선을 가지고 있어 색 충돌이 발생할 가능성이 높기 때문이다.
백트래킹은 현재 정점에 색을 배정할 때 이미 색칠된 인접 정점과 색이 같으면 해당 경우를 더 이상 탐색하지 않는다. 따라

서 간선이 많은 밀집 그래프에서는 색 충돌이 더 자주 발생하고, 그 결과 가지치기 횟수가 증가한다.

이처럼 제약 조건이 강한 그래프에서는 불가능한 경우를 더 빨리 발견할 수 있으므로 방문 노드 수가 감소할 수 있다. 반대로 희소 그래프는 간선이 적어 제약 조건이 상대적으로 약하다.

따라서 색 충돌이 늦게 발견되거나 적게 발생하여 밀집 그래프보다 가지치기 횟수는 적고 방문 노드 수는 더 많게 나타났다.

4.2 정점 번호 순서와 차수 내림차순 순서의 차이

기본 백트래킹은 정점 번호 순서대로 색을 배정하고, 개선 백트래킹은 차수가 높은 정점부터 색을 배정한다.

이번 실험에서 `sparse_coloring.txt`와 `dense_coloring.txt`는 기본 백트래킹과 개선 백트래킹의 방문 노드 수 및 가지치기 횟수가 동일하게 나타났다.

`sparse_coloring.txt`에서는 두 방식 모두 방문 노드 수가 16개, 가지치기 횟수가 9회였고, `dense_coloring.txt`에서는 두 방식 모두 방문 노드 수가 13개, 가지치기 횟수가 18회였다. 이는 두 그래프에서는 정점 번호 순서로 탐색해도 비교적 빠르게 가능한 색 배정을 찾을 수 있었기 때문으로 볼 수 있다.

하지만 `hard_coloring.txt`에서는 두 방식의 차이가 크게 나타났다.

기본 백트래킹은 3,940개의 노드를 방문하고 7,881번 가지치기를 수행하였다. 반면 개선 백트래킹은 28개의 노드만 방문하고 57번 가지치기를 수행하였다.

이 차이는 `hard_coloring.txt`가 색칠 불가능한 그래프라는 점에서 더 뚜렷하게 나타난다.

색칠 가능한 그래프에서는 하나의 올바른 색 배정을 찾으면 탐색을 종료할 수 있지만, 색칠 불가능한 그래프에서는 가능한 해가 없음을 확인해야 하므로 더 많은 탐색이 필요하다. 이때 차수가 높은 정점을 먼저 선택하면 인접 정점이 많은 정점부터 색을 배정하게 된다. 이러한 정점은 다른 정점들과의 충돌 가능성이 크기 때문에, 불가능한 색 배정을 더 이른 단계에서 발견할 수 있다.

그 결과 개선 백트래킹은 기본 백트래킹보다 방문 노드 수와 가지치기 횟수를 크게 줄일 수 있었다.

4.3 유망 함수가 건전해야 하는 이유

백트래킹에서 유망 함수는 현재까지 만들어진 부분 해가 앞으로 완전한 해로 확장될 가능성이 있는지 판단하는 역할을 한다. 그래프 색칠 문제에서는 현재 정점에 어떤 색을 배정했을 때, 이미 색칠된 인접 정점과 색이 같은지 검사하는 조건이 유망 함수에 해당한다.

유망 함수가 건전하다는 것은 해가 될 가능성이 있는 경우를 잘못 제거하지 않는다는 뜻이다.

만약 유망 함수가 건전하지 않다면, 실제로는 올바른 색칠 방법으로 이어질 수 있는 가지를 잘못 불가능하다고 판단하여 가지치기할 수 있다. 이 경우 알고리즘은 가능한 해를 탐색하지 못하고, 색칠 가능한 그래프를 색칠 불가능하다고 잘못 판단할 수 있다.

따라서 가지치기를 통해 탐색량을 줄이더라도, 정답의 정확성을 보장하려면 유망 함수는 반드시 건전해야 한다.

즉, 백트래킹에서 가지치기는 단순히 탐색량을 줄이기 위한 과정이 아니라, 정답이 될 가능성이 없는 경우만 안전하게 제거하는 과정이어야 한다.

유망 함수의 건전성은 이러한 가지치기가 정답을 잃지 않고 수행되도록 보장하는 조건이다.

5. 결론

본 과제에서는 그래프 색칠 문제를 완전 탐색, 기본 백트래킹, 개선 백트래킹의 세 가지 방법으로 해결하고 결과를 비교하였다.

완전 탐색은 모든 색 배정을 검사하기 때문에 구현은 단순하지만 방문 노드 수가 매우 많았다. 특히 `hard_coloring.txt`처럼 색칠이 불가능한 경우에는 가능한 모든 경우를 확인해야 하므로 실행 시간이 가장 길게 나타났다.

기본 백트래킹은 색을 배정하는 과정에서 인접 정점과 색이 충돌하면 더 이상 탐색하지 않고 가지치기하였다. 이로 인해 완전 탐색에 비해 방문 노드 수와 실행 시간이 크게 감소하였다.

개선 백트래킹은 차수가 높은 정점부터 색을 배정하도록 정점 선택 순서를 변경한 방법이다. `sparse_coloring.txt`와 `dense_coloring.txt`에서는 기본 백트래킹과 큰 차이가 없었지만, `hard_coloring.txt`에서는 방문 노드 수가 3,940개에서 28개로 크게 감소하였다.

이를 통해 그래프 색칠 문제에서 백트래킹은 완전 탐색보다 효율적이며, 특히 색칠이 어려운 그래프에서는 정점 선택 순서가 탐색 성능에 큰 영향을 줄 수 있음을 확인하였다. 따라서 단순히 가지치기를 적용하는 것뿐만 아니라, 어떤 순서로 상태 공간 트리를 탐색할지 결정하는 것도 알고리즘의 성능 향상에 중요한 요소라고 볼 수 있다.